

Session 15

GitHub & Remote Collaboration Workflow

What are you going to learn in this session?

In this session, you will move from working with Git locally to collaborating using remote repositories on GitHub. You will learn how to connect your local project to GitHub, push your code online, and work with repositories in a way that reflects real-world development practices.

You will also understand how developers collaborate using branches, how to clone projects, and how to keep your code updated using pull and fetch. By the end of the session, you will be able to confidently manage your code both locally and remotely.

Why is this topic important?

In real-world development, your code does not live only on your machine. It is stored on remote platforms so that teams can collaborate, review, and deploy code efficiently. GitHub is one of the most widely used platforms for this purpose.

Understanding how to push, pull, and collaborate using GitHub is essential because it connects your development workflow to the real world. Without this, you are only working in isolation, which is not how professional software development happens.

How should you prepare yourself to attend this session?

Before attending this session, ensure you are comfortable with basic Git commands such as commit, branch, and merge. Your project should already be a Git repository with a few commits.

Make sure you have access to a browser and can create accounts online, as you will be setting up GitHub during the session. Keep your terminal open and be ready to execute commands step by step.

What should you do after the session?

After the session, you should push all your projects to GitHub and start maintaining them there. Practice cloning repositories, making changes, and pushing updates regularly.

You should also begin following a simple workflow where you create branches for new features and push them to GitHub. This will prepare you for real-world team collaboration scenarios.

Class Notes

Git vs GitHub (Clear Distinction)

It is very important to clearly understand the difference between Git and GitHub. Many beginners confuse the two, but they serve different purposes in the development workflow.

Git is a version control tool that works locally on your machine and helps you track changes in your project. GitHub, on the other hand, is a cloud platform that hosts your repositories and allows you to collaborate with others.

You can think of Git as the engine that tracks changes, and GitHub as the platform where your code is stored, shared, and managed with others.

GitHub Account & Profile Setup

To start using GitHub, you need to create an account and set up a basic profile. This allows your work to be associated with your identity and makes collaboration possible.

Make sure you complete the following steps:

- Create an account and verify your email
- Set a profile name that represents you professionally
- Optionally add a profile picture

This setup ensures that your commits and repositories are properly linked to you.

Creating a Remote Repository

A remote repository is the online version of your project that will be hosted on GitHub. You will create this repository from the GitHub interface.

While creating the repository, ensure that you do not initialize it with a README file so that you can connect it cleanly with your existing local repository.

Once created, copy the repository URL, which will be used to link your local project to GitHub.

Connect Local Repo to GitHub

To connect your local repository to GitHub, you need to add a remote origin. This tells Git where to push your code.

```
Shell
git remote add origin <repo-url>
```

You can verify the connection using:

```
Shell
git remote -v
```

The term "origin" is just a default name used for the remote repository. It represents your GitHub repository.

Push Code to GitHub

Once the connection is established, you can push your local code to GitHub. This uploads your project and makes it available online.

```
Shell
git push -u origin main
```

The `-u` flag sets the upstream branch, so future pushes can be done without specifying the branch name again.

After pushing, refresh your GitHub repository page to confirm that your files are visible.

Clone a Repository

Cloning allows you to download a complete repository, including its history, from GitHub to your local machine. This is commonly used when working on shared projects.

```
Shell
git clone <repo-url>
```

This command creates a new folder with the project files and sets up the remote connection automatically.

Pull and Fetch (Keeping Code Updated)

When working with remote repositories, you need to keep your local code updated with changes made by others. This is done using pull and fetch commands.

```
Shell  
git pull
```

This command fetches the latest changes from the remote repository and merges them into your current branch.

```
Shell  
git fetch
```

This command only downloads changes without merging them, allowing you to review updates before applying them.

Collaboration Workflow Using Branches

In a team environment, developers do not work directly on the main branch. Instead, they create feature branches for their changes and then push those branches to GitHub.

```
Shell  
git switch -c feature-update  
# make changes  
git add .  
git commit -m "update feature"  
  
git push origin feature-update
```

This workflow ensures that the main branch remains stable while new features are developed independently.

Basic Collaboration Flow

In a collaborative setup, each team member works on their own branch and pushes changes to GitHub. These changes are later reviewed and merged into the main branch.

Developers frequently switch branches, pull the latest updates, and ensure their work is in sync with the team. This cycle repeats continuously during development.

The concept of pull requests will be covered later, but for now, focus on understanding how branches are created, pushed, and managed.

GitHub Desktop (Optional)

GitHub Desktop provides a graphical interface for working with Git and GitHub. It allows you to commit and push changes without using the command line.

While this can be helpful for beginners, learning the command line workflow is strongly recommended because it gives you more control and deeper understanding.

Common Beginner Confusions

Many learners get confused between local and remote repositories. A simple way to remember this is that local repositories exist on your machine, while remote repositories exist on GitHub.

Push errors are also common and usually happen due to authentication issues or mismatched branch names. Pull and fetch confusion is another frequent issue, so remember that pull includes merging while fetch does not.

Key Takeaways

You now understand how GitHub extends Git by enabling remote storage and collaboration. You have learned how to create repositories, connect local projects, push code, clone repositories, and keep your code updated.

You have also been introduced to a basic collaboration workflow using branches, which is how teams work in real-world projects. Practicing these workflows regularly will help you become comfortable with remote development environments.

At this stage, your focus should be using GitHub consistently for all your projects so that this workflow